

L1-zkEVM breakout #07

July 2026



Agenda

- Intro
- Projects updates and Q&As

Refreshed EIP-8025 merged!

Draft Standards Track: Core

EIP-8025: Optional Execution Proofs

Introducing optional execution proofs on the consensus layer

Authors Kevaundray Wedderburn (@kevaundray), Justin Drake (@JustinDrake) <justin@ethereum.org>, Ignacio Hagopian (@jsign), Han (@han0110), Francesco Risitano (@frisitano), Cody Gunton (@codygunton)

Created 2025-09-17

Discussion Link <https://ethereum-magicians.org/t/eip-optional-execution-proofs/25500>

Requires EIP-4844, EIP-6110, EIP-7002, EIP-7251, EIP-7732, EIP-7892, EIP-7928

Table of Contents

- Abstract
- Motivation
- Specification
 - Terminology
 - Specification references
 - Consensus Layer
 - Execution Layer
- Rationale
 - Path to low-resource validation
 - Build operational experience
- Backwards Compatibility
- Test Cases
- Reference Implementation
- Security Considerations
- Copyright

Abstract

Execution proofs enable consensus layer (CL) nodes to perform stateless validation of execution payloads. Altruistic *proof-generating nodes* generate execution proofs and broadcast them over the CL p2p network. *Proof-verifying nodes* consume gossiped execution proofs and check payload validity by verifying those proofs. Verification is *stateless* and constant time with respect to the payload's gas limit and EL state size, so a validator's payload-verification requirements are decoupled from the linear cost of re-execution. The mechanism is fully opt-in and does not change consensus validity rules, allowing the proving stack to mature in production; a separate, future EIP could subsequently propose making execution proofs mandatory once they have matured. This EIP does not introduce incentives for proof-generating nodes; for that reason, they are considered altruistic and the mechanism is opt-in.

Motivation

This specification block describes the motivation for the optional execution proofs (EIP) to the following content.

Refreshed EIP-7709 merged

Draft

Standards Track: Core

EIP-7709: Read BLOCKHASH from Storage and Update Cost

Read the `BLOCKHASH (0x40)` opcode from the EIP-2935 system contract storage and adjust its gas cost to reflect storage access.

Authors Vitalik Buterin (@vbuterin), Tomasz Stanczak (@tkstanczak), Guillaume Ballet (@gballet), Gajinder Singh (@g11tech), Tanishq Jasoria (@tanishqjasoria), Ignacio Hagopian (@jsign), Jochem Brouwer (@jochem-brouwer), Gabriel Rocheleau (@gabrocheleau)

Created 2024-05-18

Discussion Link <https://ethereum-magicians.org/t/eip-7709-read-blockhash-opcode-from-storage-and-adjust-gas-cost/20052>

Requires EIP-2935

Table of Contents

- Abstract
- Motivation
- Specification
 - Activation
 - Gas costs
 - Reading from the System contract
- Rationale
- Backwards Compatibility
- Test Cases
- Security Considerations
- Copyright

Abstract

Update the `BLOCKHASH (0x40)` opcode to read and serve from the system contract storage and charge the **additional** (cold or warm) storage costs.

Motivation

The `BLOCKHASH (0x40)` opcode currently assumes that the client has access to recent block history. This makes it a protocol special case: it depends on historical chain data, but it is not modeled like other state-backed reads.

With EIP-2935, recent block hashes are stored in the system contract storage. This allows in-window `BLOCKHASH` lookups to be modeled as storage-backed accesses while preserving the existing `BLOCKHASH` return-value semantics.

Specification

<https://hackmd.io/@isign/heqota-7709>

zkEVM project 1 & 2

Execution Witness & Guest Program specs/test

Specs & tests

- zkevm-tests@v0.6.{[0](#), [1](#), [2](#)}
 - Based on *glamsterdam-devnet-7*
 - Removed *blob-schedule* and *fork activation* from *ChainConfig*
 - External t8n filling bug fixes
 - Fixes EIP-8282 new execution request not being considered
- Upcoming (soon!):
 - Make *SCHEMA_ID* part of public input.
 - Remove *ChainConfig* and simplify to only *chain_id*.
 - Progressive SSZ for ``state``, ``code`` and ``pub_keys``.
 - All benchmarks being fillable with Geth
 - *glamsterdam-devnet-8* support

Devnet 7 stateless input R2 bucket

glamsterdam-devnet-7 stateless inputs

Batch-first public dataset catalog generated at 2026-08-04T18:41:58.047620366Z.

Block artifacts

50043

Batches

4988

Batch size

10

Total batch size

33.69 GiB

How to download

Each archive contains benchmark-ready EEST fixtures under `blockchain_tests/` and metadata at `.meta/manifest.json`.

```
curl -LO exports/batches/45330-45339.tar.zst
tar --zstd -xf 45330-45339.tar.zst
```

Pass the extracted directory directly to `ere-hosts --input` folder.

Verify downloads with [SHA256SUMS](#).

Machine-readable indexes

- [manifest.json](#) Dataset manifest
- [batches.jsonl](#) Batch index

R2 public buckets do not provide directory listing; use these files instead of folder URLs.

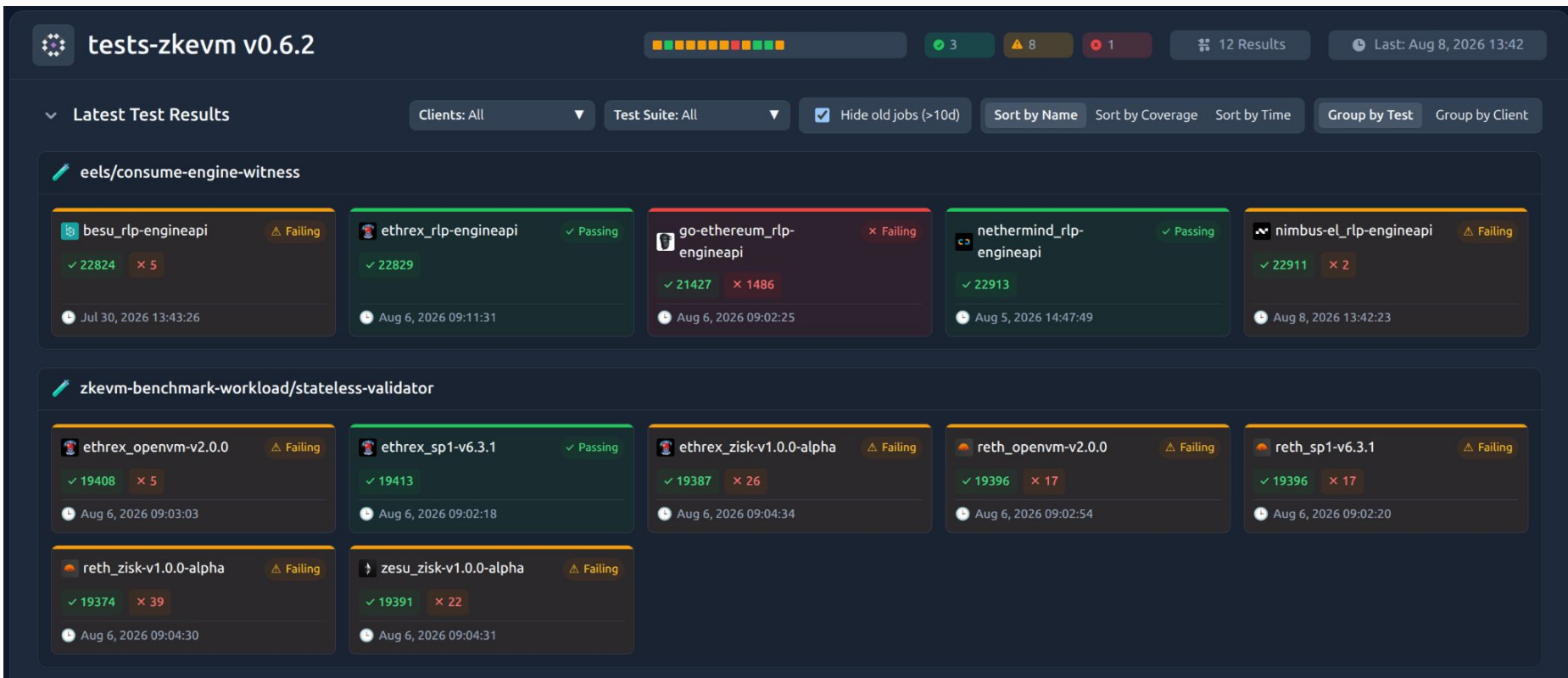
Batch archives

Blocks	Artifacts	Size	SHA-256	Download
115260-115269	10	8.42 MiB	0x30562c14a0ac7688...	115260-115269.tar.zst
115250-115259	10	7.38 MiB	0x65fef4264c7f520d...	115250-115259.tar.zst
115240-115249	10	6.94 MiB	0xe4ad1f118395c8c8...	115240-115249.tar.zst
115230-115239	10	6.99 MiB	0xb8a47449501f1b32...	115230-115239.tar.zst
115220-115229	10	5.91 MiB	0xc02b704067bec2d0...	115220-115229.tar.zst
115210-115219	10	7.03 MiB	0xb0d74a73a18e76d9...	115210-115219.tar.zst
115200-115209	10	6.75 MiB	0xdef3a876f3de0ad1...	115200-115209.tar.zst

<https://pub-df22334654034ebab51bc096137a59d8.r2.dev/devnets/glamsterdam-devnet-7/index.html>

Hive-dashboard

<https://eth-act.github.io/eest-execution-witness-dashboard/#/group/tests-zkevm%20v0.6.2>



Thanks all stateful-EL teams for adjusting code to be included! In particular FLCL, Rubo and Deme for last minute changes!

Guest program handbook!

This document defines the criteria used by the EF Research Engineering (zkEVM project) to assess a guest program's readiness for inclusion across our tools, benchmarks, and overall consideration for EIP-8025. These guidelines represent our recommendations to All Core Devs (ACD) within our team's capacity, and should not be interpreted as authoritative protocol rules for Ethereum.

Guest program rubric

We describe the rubric by which we evaluate guest programs' readiness:

- Must implement the EEST specs and [pass all tests](#).
- The ELF must comply with the defined [standards](#).
- The ELF and Verification Key must be released as a signed asset in the guest program public repository releases, generated by a public, fully open-source CI pipeline.
- Regarding ELF generation:
 - Modified/Custom compiler: Formal verification (FV) requirements will apply to the compiled ELF (TBD by the EF FV project team).
 - No Compiler (e.g., raw assembly/manual bytecode): The raw ELF artefact will be subject to a higher degree of formal verification scrutiny (TBD by the EF FV project team).
- The repository license must be MIT + Apache 2.0.
- The team maintaining the guest program must be considered *sustainable*. While this is hard to define precisely, preference is given to guest programs maintained by established execution-layer teams or credible working groups.
-

More details about some of the above points are in the following sections.

EEST specs and tests

What the guest program should do is defined in [execution-specs](#) repository under the `src/ethereum/forks/amsterdam` folder. The repository has releases with name `tests-zkevm@vA.B.C` ([example](#)).

The releases are tags on `projects/zkevm-releases` branch, which is a merge of two things:

1. `projects/zkevm` which is the main branch where we do zkevm-related spec/tests changes, which is on top of `execution-specs` main branch (today named `forks/amsterdam`).
2. `tests-<fork>-devnet-vA.B.C` some other branch related to a devnet was launched (e.g. `tests-glamsterdam-devnet@v6.1.0`).

Refer to the latest `tests-zkevm@vA.B.C` tag to view the canonical specs and tests. If you have questions about guest program implementation or want to propose changes, open an issue in the `execution-specs` repository.

To run tests, the releases contain a `.tar.gz` with usual EEST fixtures with extra fields:

- `statelessInputBytes` : are the bytes that **must** be received in the guest program. No host manipulation of these bytes is allowed, as it can introduce new bugs/vulnerabilities not covered by EEST tests.
- `statelessOutputBytes` : are the bytes the guest program is expected to write as public input.
- `executionWitness` : for guest programs is an auxiliary field that provides more details of the execution witness encoded in the `statelessInputBytes`. Only meant for debugging reasons.

Release ELFs in teams GH repo via CI

The ELF build for running in zkVMs should be the output of a public, fully open-source CI pipeline in your repository.

The guest program teams must publish final ELFs with proper signatures for the ELF and the Verification Keys. The zkEVM team will facilitate in [ere-guests](#) some scripts/tools to do the final linking against target zkVMs which provide libraries for [zkevm-standards](#).

The ELF expected naming must be: `stateless-validator-<guestName>-<zkVMName>-<zkVMVersion>.elf` The VK expected naming must be: `stateless-validator-<guestName>-<zkVMName>-<zkVMVersion>.vk` (refer to [this document](#)).

In `ere-guests`, we might republish ELFs of many guest program repos verbatim, just to have a hub of all ELFs targeted for 8025. But as mentioned, no real linking, re-signing or similar will be done; the source of truth for ELFs is always the guest program teams' repository releases.

<https://github.com/eth-act/zkevm-standards/blob/main/handbooks/zkvm-handbook.md>

zkEVM project 6

Benchmarking and Metrics



Zkevm-benchmark-workload + ere-guests

- [zkevm-benchmark-workload](#)
 - Unified unique fixture format (i.e. EEST fixtures) both for tests and benchmarks
 - Improved R2 bucket collector (*witness-generator-spec-cli* daemon).
 - Add *opcode_count* and *target_opcode* into fixture metadata
- [ere-guests](#)
 - Leverages unified fixture format mentioned above
 - Moving more into ELF-hub architecture
 - Runs in CI: EEST fixtures, Devnet-7 R2 bucket, and mainnet blocks

More updates

Announcing the OpenVM 2.1 Preview Release

Published Jul 29, 2026

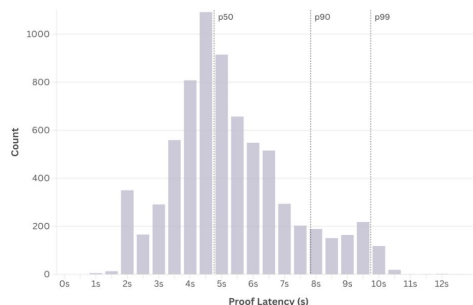
We are announcing OpenVM 2.1 Preview, an early look at the next version of OpenVM. This release adds 64-bit RISC-V support to OpenVM and introduces full compatibility with the Ethereum Foundation's proposed RV64IM_Zicclsm target for zkVMs. It also introduces substantial performance improvements driven by a new execution approach based on portable native recompilation, allowing OpenVM 2.1 Preview to prove mainnet blocks in real time on only 4 GPUs, a 2x improvement compared to OpenVM 2.0.

OpenVM 2.1 Preview is available under MIT and Apache 2.0 dual-license on [GitHub](#). We are shipping this preview release now to give a snapshot of ongoing performance improvements for OpenVM and to allow developers to start building against the 64-bit toolchain. It has not gone through our production security pipeline including audits or formal verification, and we continue to recommend [OpenVM 2.0](#) for production use.

<https://blog.openvm.dev/2.1>

Real-Time Ethereum Proving on 4 GPUs

OpenVM 2.1 Preview proves Ethereum mainnet blocks in real time (defined as proving blocks within Ethereum's block time) on 4 5090 GPUs, with a p99 proving time of 9.7s, an average of 5.0s, and a max of 12.1s over 7,200 blocks starting from block 24,000,000. This cuts hardware requirements in half from [OpenVM 2.0](#), making real-time proving more accessible.



Benchmarks run on 4 bare-metal RTX 5090 GPUs

On 16 GPUs, OpenVM 2.1 Preview delivers a p99 time of 3.7s and an average of 2.4s. This represents a roughly 1.7x improvement over [OpenVM 2.0](#), which proves at a p99 time of 6.3s over the same stretch of blocks. We are demonstrating the performance of this 16-GPU cluster live on [EthProofs](#), submitting proofs for every mainnet block.

Metric	OpenVM 2.1 Preview	OpenVM 2.0
Average	2.4s	3.9s
p50 time	2.4s	3.8s
p90 time	3.1s	5.4s
p99 time	3.7s	6.3s
Max time	4.3s	7.4s

Benchmarks run on 16 bare-metal RTX 5090 GPUs